Science-Aware Retrieval Operators for AI-Driven HPC Data Discovery

Author A and Author B
Department of Computer Science
University Name, City, Country
{authora, authorb}@university.edu

***Abstract*—Scientific data at HPC facilities is scattered across filesystems, object stores, and binary formats such as HDF5 and NetCDF, with measurements expressed in inconsistent unit prefixes. A query for pressures between 190–360 kPa should match "200000 Pa" and "0.2 MPa," yet retrieval systems treat these as unrelated strings. Embedding models achieve only 0.54 accuracy on numerical content, and string normalization cannot bridge SI prefixes.**

We present clio-agentic-search, a retrieval engine that introduces *science-aware operators*—deterministic retrieval primitives that execute as first-class branches alongside BM25 and dense vector search. A dimensional-conversion operator canonicalizes physical quantities to base SI units via explicit arithmetic (kPa $\times$ 10^3 = Pa), guaranteeing correctness by construction within 13 supported units across 5 domains. A formula-normalization operator matches equations across notation variants. These operators run within a federated architecture that queries six backend types through per-backend capability negotiation, with an LLM-driven agentic loop for iterative multi-hop refinement.

We evaluate on a controlled benchmark of 210 documents across seven scientific domains with deliberate cross-unit variation and 80 queries spanning cross-unit, same-unit control, formula, and multi-constraint conditions. On cross-unit queries, the dimensional operator achieves 0.99 P@5 and 0.99 MRR where the strongest text baseline scores 0.43 P@5 and 0.49 MRR. Formula normalization provides complementary value through strict normalized matching, and the two operators compose independently. Federated search nearly doubles coverage from 24.4% to 47.2% across namespaces. An ablation study confirms that the science-aware branch contributes the largest marginal gain (+0.63 P@5). All code and benchmarks are open-source.

I. Introduction

Scientific computing generates data at unprecedented scale, yet an estimated 60–80% of that data is never reused after creation—not because it is valueless, but because it is unsearchable. When a scientist at a DOE facility cannot find that a 200 kPa combustion simulation already exists—stored as "200000 Pa" on a different filesystem—they re-run it, wasting allocations that cost millions of dollars per year per leadership-class machine. The consequences of unit mismatch extend beyond inconvenience: NASA's loss of the $327M Mars Climate Orbiter, caused by a pound-seconds versus newton-seconds confusion, remains the canonical example. The FAIR data principles now mandated by NSF, NIH, and DOE require that data be *Findable*—yet unit heterogeneity across storage tiers systematically violates this requirement. A query for pressures between 190–360 kPa should match "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3-hosted dataset. These describe identical pressure. Every retrieval system treats them as unrelated strings.

This data discovery crisis is intensifying as AI agents enter scientific workflows. Coscientist [1] autonomously plans and executes wet-lab synthesis. AI Scientist v2 [2] generates and writes up research end-to-end. Autonomous agents at ORNL orchestrate experiments across national facilities [3], and the Model Context Protocol now wraps Globus search, compute, and storage under a unified agent interface for HPC [4]. The trajectory is clear: agents are transitioning from processing scientific literature to operating scientific infrastructure. Yet these agents consistently fail at data retrieval, not reasoning. ScienceAgentBench [5], a benchmark of 44 tasks drawn from peer-reviewed publications, reports that the best agent achieves only 32.4% end-to-end success, with failures traced to data handling rather than reasoning. MLAgentBench [6] observes the same pattern. A comprehensive survey of agentic AI [7] identifies data management as explicitly underexplored. These agents can reason about science—they cannot find the data they need to reason about.

The root cause is structural: retrieval systems treat scientific content as plain text. Three compounding failures explain why.

Dimensional blindness. Scientific measurements are routinely expressed in different unit prefixes. String-based quantity normalization [8] maps unit names to canonical strings, but "kilopascal" and "pascal" remain different strings—no string operation bridges the SI prefix gap. Learned embeddings such as CONE [9] encode numbers and units in a shared vector space, but the Numeracy Gap study [10] evaluates 13 embedding models and finds 0.54 binary retrieval accuracy on numerical content—effectively random. NC-Retriever [11] reports 16.3% dense retrieval accuracy on numeric constraint queries. The fundamental problem is that none of these approaches performs the required arithmetic: $200 \times 1000 = 200{,}000$.

Formula opacity. "$F = ma$," "$F = m \cdot a$," and "$ma = F$" encode the same physical law but produce distinct token sequences. Math-aware search systems [12], [13] match equation structure in LaTeX markup but cannot connect a matched formula to the physical measurements it governs. To our knowledge, no retrieval system combines formula normalization with quantity-aware dimensional search.

Storage fragmentation. HPC data spans parallel filesystems, S3 object stores, vector databases, and binary formats

such as HDF5 and NetCDF. To our knowledge, no system provides unified search with science-aware operators across these backends. RAGRoute [14] federates generic text search. PROV-IO+ [15] captures HDF5 provenance with under 3.5% overhead but exposes no search interface.

These failures compound. The motivating query—"find experiments where pressure was between 190–360 kPa across all storage"—requires dimensional conversion across SI prefixes, federated search across heterogeneous backends, and potentially formula context linking pressure to related variables. To our knowledge, no existing system addresses even two of these three requirements simultaneously.

We argue that these failures share a common root cause: retrieval pipelines lack a mechanism for incorporating domain knowledge as executable, verifiable computation. BM25 matches tokens. Dense models match embeddings. Neither can encode the fact that $5\ \text{km} = 5000\ \text{m}$—an algebraic invariant, not a statistical similarity. We introduce *science-aware operators*: deterministic, pluggable retrieval primitives that encode domain knowledge as explicit arithmetic and symbolic transformations, executing as first-class branches alongside standard BM25 and dense vector search. The architecture is general—any domain where correctness is non-negotiable (chemical identifiers, genomic coordinates, engineering tolerances) can be served by operators of this class. We instantiate the framework for scientific measurement retrieval and test it through five contributions:

1) **Dimensional-conversion retrieval.** We canonicalize physical quantities to base SI units via arithmetic multiplication (e.g., kPa $\times$ 10^3 = Pa) across 13 units in 5 domains, with correctness guaranteed by construction, provided units are correctly extracted and conversion factors are complete.
2) **Formula normalization.** We normalize mathematical expressions across whitespace, superscript, side-swap, and factor-reordering variants, integrated with dimensional operators as parallel retrieval branches.
3) **Federated multi-namespace search.** A connector architecture supports unified retrieval across filesystem, S3, Qdrant, Neo4j, HDF5, and NetCDF backends with per-backend capability negotiation and incremental indexing.
4) **HDF5/NetCDF metadata indexing.** Dedicated connectors extract measurements from scientific file formats and feed them through the same SI canonicalization pipeline, enabling cross-format dimensional search.
5) **Multi-hop agentic retrieval.** An LLM-driven query rewriting loop iteratively refines searches using expand, narrow, and pivot strategies, converging within 2–3 hops.

The remainder of this paper is organized as follows. Section 2 surveys hybrid retrieval, the numeracy crisis, quantity-aware retrieval, and scientific data discovery. Section 3 presents the system design. Section 4 describes the implementation. Section 5 evaluates across dimensional conversion, formula matching, federated search, and ablation. Section 6 concludes.

II. Background and Related Work

The gap between what retrieval systems deliver and what scientific data discovery demands spans four areas: the hybrid retrieval foundations we build upon, the numeracy crisis that motivates explicit arithmetic operators, quantity-aware retrieval systems that attempt but fail to solve cross-unit matching, and scientific data discovery in HPC.

A. Hybrid Retrieval Foundations

Modern retrieval combines lexical and dense signals. BM25 [16] remains the strongest lexical baseline; dense retrieval via learned bi-encoders [17] captures semantic similarity that keyword matching misses. Bruch et al. [18] show that a learned convex combination consistently outperforms reciprocal rank fusion (RRF) on TREC benchmarks, establishing the analytic framework we adopt. At the agentic frontier, A-RAG [19] introduces hierarchical retrieval interfaces that let LLM agents compose retrieval strategies dynamically.

We build *on* hybrid retrieval, not beside it. Our science-aware operators execute as parallel branches alongside BM25 and dense search within the same fusion pipeline, adding domain-specific recall without replacing any existing component.

B. The Numerical Reasoning Crisis

Numbers are fundamentally broken in both retrieval systems and large language models. The Numeracy Gap study [10] evaluates 13 embedding models on numerical content and reports 0.54 binary retrieval accuracy—barely above random chance. NC-Retriever [11] finds that dense retrieval achieves only 16.3% accuracy on queries with numeric constraints, even when the constraint is stated explicitly in both query and document. The root cause is architectural: tokenizers encode numbers per-digit rather than by magnitude, so "200" and "200000" share no meaningful representation [20]. GSM-Symbolic [21] delivers the sharpest result: changing only the numeric values in grade-school math problems while preserving problem structure causes a 65% accuracy drop, proving that LLMs pattern-match on surface numerals rather than reason about magnitude.

This is not merely a unit problem. Numbers themselves—the substrate of all scientific measurement—are broken in every retrieval and language model. Scientific data retrieval, where matching requires comparing magnitudes across units, is especially vulnerable.

C. Quantity-Aware Retrieval

Three lineages attempt to make retrieval quantity-aware. None succeeds at arithmetic conversion.

String normalization. QFinder [22] extends Elasticsearch with quantity-aware ranking by extracting and normalizing numeric expressions. CQE [23] advances this with dependency parsing over a 531-unit dictionary, handling complex quantity expressions. Numbers Matter! [8] builds a full retrieval pipeline with quantity-aware indexing and evaluation

on FinQuant and MedQuant benchmarks. The fundamental limitation persists: string normalization cannot cross SI prefixes. "kilopascal" and "pascal" remain distinct strings after normalization, so a query for "200 kPa" never matches a document containing "200000 Pa."

Learned embeddings. CONE [9] trains a transformer to embed number–unit pairs into a shared vector space, achieving a 25% Recall@10 gain on quantity-bearing queries. NC-Retriever [11] applies contrastive learning to numeric constraint matching. These approaches are probabilistic: a model may learn that 200 kPa is *similar* to 200000 Pa, but provides no correctness guarantee. Weller et al. [24] prove theoretical upper bounds on embedding-based retrieval, showing that fixed-dimensional embeddings cannot faithfully preserve all distance relationships—a result that undercuts learned approaches to exact quantity matching.

Math-aware search. Approach0 [12] matches equation structure via operator trees. SSEmb [13] learns semantic similarity over mathematical expressions. These systems match formula *structure* but are disconnected from physical measurements—they cannot link $F = ma$ to a document reporting force in newtons.

Gap. To our knowledge, no existing retrieval system performs arithmetic SI conversion to canonicalize measurements. Nor does any system combine quantity matching with formula normalization or operate across federated HPC backends with science-aware operators.

D. Scientific Data Discovery in HPC

Paper search. OpenScholar [25] indexes 45 million open-access papers and achieves GPT-4o-level citation quality. HiPerRAG [26] scales RAG to 3.6 million articles across DOE supercomputers Polaris, Sunspot, and Frontier. Both search *papers*—not experimental data, simulation outputs, or HDF5 files.

Data search. Recent systems do search scientific data. PANGAEA-GPT [27] deploys hierarchical multi-agent search over 400,000+ geoscientific datasets with data-type-aware routing and unit scale validation, achieving 8.14/10 relevance versus a 2.87/10 baseline. LLM-Find [28] uses LLM-generated code to retrieve geospatial data from heterogeneous sources with 80–90% success. The distinction is temporal: PANGAEA-GPT validates unit scales *post-hoc* (after retrieval); we convert *at retrieval time* (during indexing and query processing). LLM-Find generates access code per source; we provide reusable retrieval operators.

HPC data management. Datum [29] catalogs scientific data at INL but offers no content search. BRINDEXER [30] indexes POSIX metadata on Lustre with 69% query improvement—metadata only, no content. PROV-IO+ [15] captures HDF5 I/O provenance with under 3.5% overhead but exposes no search interface. HDF Clinic [31] proposes a knowledge graph over HDF5 but remains unimplemented.

Agentic retrieval. The SoK on Agentic RAG [32] provides the first systematization of knowledge in this space. Context-1 [33] demonstrates 20-billion-token agentic search.

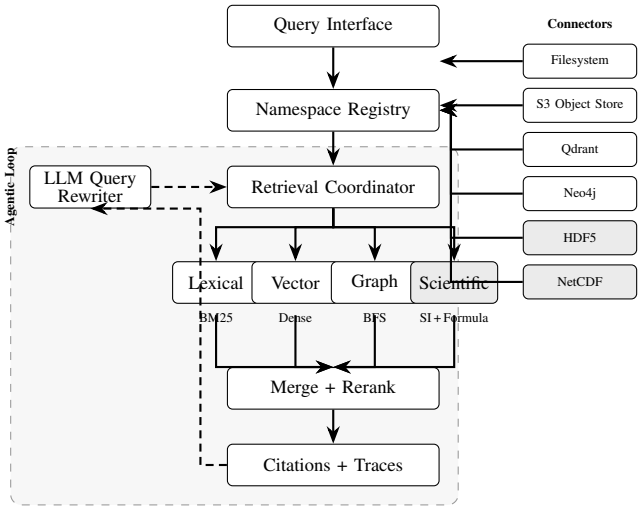

Fig. 1. System architecture of CLIO-AGENTIC-SEARCH. Science-aware components (shaded) execute alongside standard retrieval branches. The agentic loop iteratively refines queries via LLM rewriting.

CLADD [34] augments collaborative drug-discovery agents with domain knowledge graph operators. SciAgent [35] coordinates specialist worker agents under a hierarchical controller. ScienceClaw [36] deploys a 300+ skill agent swarm with immutable artifact provenance. All improve orchestration. To our knowledge, none incorporates science-aware numerical operators—arithmetic dimensional conversion or formula normalization—into the retrieval pipeline itself.

Table I summarizes the landscape. To our knowledge, no prior system supports the combination of our first five capabilities: arithmetic SI conversion, formula normalization, science-aware retrieval branches, federated multi-backend search, and HDF5/NetCDF indexing. Individual capabilities may exist in isolation. We honestly mark capabilities we lack—we do not index papers. PANGAEA-GPT receives partial credit for quantity extraction (unit scale validation, but post-hoc rather than at retrieval time).

III. System Design

To address the three failures identified in Section 1, we design clio-agentic-search around a central principle: science-aware operators must execute as first-class retrieval branches, not post-processing filters. The architecture comprises a staged pipeline with four parallel branches (Sec. 3.1), dimensional conversion operators that perform explicit SI arithmetic (Sec. 3.2), deterministic formula normalization (Sec. 3.3), a hybrid fusion strategy (Sec. 3.4), federated multi-namespace search (Sec. 3.5), and an LLM-driven multi-hop retrieval loop (Sec. 3.6).

A. Architecture Overview

The system is a staged pipeline with four parallel retrieval branches (Fig. 1). A query enters through the Query Interface (FastAPI REST endpoint or CLI), which parses the raw query string, extracts typed scientific constraints (numeric ranges,

TABLE I
CAPABILITY COMPARISON ACROSS RETRIEVAL AND SCIENTIFIC DATA SYSTEMS. ✓ = SUPPORTED, ✗ = NOT SUPPORTED, ○ = PARTIAL.

	Our Contributions					General Capabilities				
System	SI Conversion	Formula Norm.	Sci. Branches	Federated	HDF5/NetCDF	Agentic	Qty. Extract.	Paper Search	Data Search	LLM Integr.
Ours (clio)	✓	✓	✓	✓	✓	✓	✓	✗	✓	✓
PANGAEA-GPT	✗	✗	✗	✗	✗	✓	○	✗	✓	✓
OpenScholar	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
HiPerRAG	✗	✗	✗	✗	✗	✗	✗	✓	✗	✓
Numbers Matter!	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CONE	✗	✗	✗	✗	✗	✗	✓	✗	✗	✗
CLADD	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
SciAgent	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓
Context-1	✗	✗	✗	✗	✗	✓	✗	✗	✗	✓

unit filters, formula patterns), and forwards a structured query object to the Namespace Registry. The registry resolves which namespaces—each backed by one of six connector types—are eligible for the query based on capability negotiation (Sec. 3.5).

The Retrieval Coordinator fans the query out to four parallel branches: lexical (BM25), vector (dense embedding similarity), graph (entity-relationship traversal), and scientific (dimensional measurement and formula queries). Each branch executes independently and returns scored candidate chunks. The Merge and Rerank stage deduplicates candidates by chunk identifier, applies hard scientific filters, computes a composite score via heuristic fusion, and emits the final top-K results with source citations.

Six connectors mediate access to heterogeneous backends: local filesystem, S3-compatible object stores, Qdrant vector database, Neo4j graph database, HDF5 (via h5py), and NetCDF (via xarray). Each connector declares a capability set specifying which retrieval branches it supports, enabling the coordinator to skip unsupported branches rather than fail. Every pipeline stage emits structured trace events for observability.

B. Dimensional Conversion Operators

The core insight behind dimensional conversion is that cross-unit matching is an arithmetic problem, not a similarity problem. When a scientist queries for pressures between 190 and 360 kPa, the system must recognize that a document reporting "250000 Pa" falls within that range. This requires multiplying 190 by 1000 to obtain 190000 Pa and 360 by 1000 to obtain 360000 Pa—a computation that no string normalization or learned embedding can guarantee.

We define a conversion table covering five SI domains and 13 units (Table II). Each entry maps a source unit to a canonical base unit and a multiplicative conversion factor. At index time, a regex-based measurement extractor identifies numeric-unit pairs in document text and scientific file metadata. Each extracted measurement is stored as a 4-tuple: (raw_value, raw_unit, canonical_value, canonical_unit). For example, "200 kPa" yields (200.0, "kPa", 200000.0, "Pa"). These tuples populate a `scientific_measurements` table in DuckDB, indexed on canonical_value and canonical_unit for efficient range scans.

TABLE II
SI CONVERSION TABLE: 5 DOMAINS, 13 UNITS. EACH SOURCE UNIT MAPS TO A CANONICAL BASE UNIT VIA A MULTIPLICATIVE FACTOR.

Domain	Source Units	Base	Conversion Factors
Distance	mm, cm, m, km	m	$\times 10^{-3}$, $\times 10^{-2}$, 1, $\times 10^{3}$
Mass	mg, g, kg	kg	$\times 10^{-6}$, $\times 10^{-3}$, 1
Time	s, min, h	s	1, $\times 60$, $\times 3600$
Pressure	Pa, kPa, MPa	Pa	1, $\times 10^{3}$, $\times 10^{6}$
Velocity	m/s, km/h	m/s	1, $\times \frac{1}{3.6}$

At query time, three operators execute against the canonicalized measurement store. The **numeric-range** operator parses range expressions of the form `low:high:unit`, converts both bounds to canonical units, and issues a DuckDB range predicate: a query "190:360:kPa" becomes a scan for canonical_value between 190000 and 360000 with canonical_unit = "Pa". The **unit-match** operator filters chunks containing any measurement in a specified unit domain regardless of value. The **value-match** operator retrieves measurements matching a specific canonical value within a configurable tolerance (default $\pm 1\%$).

The correctness guarantee distinguishes this approach from all prior work. Consider matching "200 kPa" against "200000 Pa". Arithmetic conversion computes $200 \times 1000 = 200000$ and succeeds by exact equality. String normalization maps

"kilopascal" to one string and "pascal" to another—they never match. Learned embeddings [9] encode the two quantities as vectors in a shared space, but the Numeracy Gap study [10] reports 0.54 mean binary retrieval accuracy across 13 embedding models on numerical content, effectively random against a 0.50 baseline. Our operators are correct by construction: if the SI conversion factor is correct, the match is correct. We note that this guarantee depends on two assumptions: (1) the regex-based extractor correctly identifies numeric-unit pairs in the source text, and (2) the conversion table covers the unit in question. Our prototype supports 13 units across 5 domains; extending coverage requires only adding entries to the conversion table, not modifying the architecture.

C. Formula Normalization

Scientific formulas appear in inconsistent surface forms across documents. "$F = m*a$", "$F = ma$", "$F = m \cdot a$", and "$ma = F$" express the same physical law but produce distinct token sequences that defeat both keyword and embedding retrieval. We normalize formulas through a deterministic six-step pipeline.

First, all whitespace is stripped. Second, all alphabetic characters are lowercased. Third, superscript notations are unified: LaTeX-style `^{N}` and Python-style `**N` are both normalized to `^N`. Fourth, the expression is split on the equality sign into left-hand and right-hand sides. Fifth, within each side, multiplicative factors are sorted lexicographically, so that `m*a` and `a*m` produce the same canonical form. Sixth, the two sides are rejoined with a normalized equality separator. The canonical form of all four variants above is `a*m=f`.

Normalized formulas are stored in a `scientific_formulas` table in DuckDB alongside their source chunk identifiers and the raw original text. At query time, the user's formula expression passes through the same six-step pipeline before lookup. Formula matching integrates with the dimensional conversion operators as a parallel branch in the scientific retrieval stage: a query can simultaneously request formulas containing a specific equation and measurements within a numeric range, and both constraints execute as independent DuckDB queries whose results are intersected at the merge stage.

D. Hybrid Retrieval Pipeline

The retrieval pipeline executes four branches in parallel and merges their results through a deterministic fusion strategy.

Lexical branch. BM25 with parameters $k_1 = 1.2$ and $b = 0.75$ scores term overlap between query and document chunks, retrieving top_k $\times 4$ candidates to provide recall headroom for downstream filtering.

Vector branch. Chunks are embedded with MiniLM-L6-v2 (384-dimensional vectors, cosine similarity) and indexed in Qdrant. This branch retrieves top_k $\times 4$ candidates.

Graph branch. For Neo4j-backed namespaces, breadth-first traversal at depth 1 from query entities retrieves related chunks through typed relationships (e.g., cites, measures, uses_formula). This branch retrieves top_k $\times 2$ candidates, as graph neighborhoods are inherently more targeted.

Scientific branch. The dimensional conversion and formula normalization operators (Secs. 3.2–3.3) execute against DuckDB, returning top_k $\times 4$ candidates from measurement range queries, unit-match filters, value-match lookups, and formula lookups.

Merge and rerank. Candidates from all branches are deduplicated by chunk_id, retaining the maximum per-branch score. A scientific hard filter then applies: if the query specifies a numeric range or unit constraint, candidates not returned by the scientific branch are discarded—dimensional correctness is enforced, not merely preferred. A metadata filter removes candidates violating explicit metadata constraints (e.g., date range, file type). Surviving candidates are scored by a configurable weighted sum of branch scores (lexical, vector, metadata, scientific), with weights tunable per deployment. The top-K candidates by composite score constitute the final result set. Each result carries provenance: the source namespace, chunk identifier, file path, and the set of branches that contributed to its retrieval.

E. Federated Multi-Namespace Search

Scientific data at HPC facilities spans heterogeneous backends. A single experimental campaign may deposit raw output on a parallel filesystem, preprocessed arrays in HDF5, derived features in a vector database, and annotations in a knowledge graph. Rather than require data migration, we bring the retrieval operators to the data.

Every storage backend is wrapped by a connector that implements the `NamespaceConnector` protocol and declares a `RetrievalCapabilities` object specifying which retrieval branches it supports. The capability matrix is: Filesystem and S3 connectors support lexical, vector, and scientific search; Qdrant supports vector search; Neo4j supports graph traversal; HDF5 and NetCDF connectors support metadata and scientific search.

The Retrieval Coordinator consults this matrix before dispatching. When a query spans multiple namespaces, the coordinator executes the single-namespace pipeline independently for each namespace, activating only the branches that the namespace's connector supports. Results from all namespaces are collected into a global candidate pool, sorted by composite score, deduplicated by chunk_id (since the same document may appear in multiple backends), and truncated to the final top-K.

Incremental indexing avoids redundant reprocessing. Each connector tracks indexed documents by SHA-256 content hash and modification time. On re-index, only documents whose hash or mtime has changed are reprocessed, enabling efficient index maintenance over evolving corpora without full rebuild.

F. Multi-hop Agentic Retrieval

Single-pass retrieval often fails for complex scientific questions that require iterative refinement. The multi-hop agentic retrieval loop automates this process.

The loop proceeds as follows. The initial query executes the full hybrid pipeline described in Section 3.4 across all federated namespaces. The retrieved results, along with the original query and the retrieval trace, are passed to an LLM (Anthropic API) that evaluates result quality and selects one of four strategies: **expand** (broaden the query to increase recall), **narrow** (add constraints to increase precision), **pivot** (reformulate the query to pursue a different aspect), or **done** (terminate the loop). If the strategy is not done, the LLM generates a rewritten query, which re-enters the pipeline for another full retrieval pass. Results from all hops are merged, deduplicated, and re-ranked. The loop terminates when the LLM selects done, when results stabilize across consecutive hops (no new unique chunk_ids appear), or when a configurable maximum hop count is reached.

Each hop constitutes a full pipeline execution: all four retrieval branches, all eligible namespaces, merge and rerank. This ensures that every rewritten query benefits from the same dimensional conversion, formula normalization, and federated search capabilities as the original. The LLM's role is strictly advisory—it selects rewriting strategies and generates query refinements but never scores, filters, or overrides the deterministic science-aware operators, preserving correctness guarantees regardless of LLM availability. When the LLM is unavailable, the system falls back to deterministic SI unit expansion.

IV. Implementation

clio-agentic-search is implemented as a single-binary Python system with no external service dependencies beyond an optional LLM endpoint. We cover the software stack and storage schema (Sec. 4.1), connectors for binary scientific formats (Sec. 4.2), and the experimental platform (Sec. 4.3).

A. System Implementation

The system is implemented in Python 3.11+ and exposes both a CLI and an async HTTP API via FastAPI. The CLI (`clio`) and HTTP API expose all retrieval modes including `--agentic`, `--numeric-range`, and `--formula` flags.

Dense retrieval uses sentence-transformers with the all-MiniLM-L6-v2 model (22.7M parameters, 384-dimensional embeddings, Apache 2.0 license). Two ANN backends are supported behind a common `ANNAdapter` protocol: `ExactANNAdapter`, which partitions vectors across 16 shards and computes exact cosine similarity, and `HnswANNAdapter`, which delegates to hnswlib for approximate search on larger corpora. Write safety on shared filesystems is enforced via `fcntl` file locking. Observability is provided by OpenTelemetry instrumentation with Prometheus-compatible `/metrics` endpoints; all retrieval branches and agentic hops emit structured trace events.

All persistent state resides in a single DuckDB database file. DuckDB was selected for three reasons: it is embedded (no external service to deploy on compute nodes), it supports SQL-native full-text search sufficient for BM25 scoring, and its single-file storage model is compatible with POSIX parallel filesystems common in HPC environments. The schema comprises eight tables (Table III).

TABLE III
DuckDB storage schema (8 tables). Science-aware tables (shaded) enable dimensional and formula retrieval.

Table	Purpose	Key Columns
documents	Corpus records	URI, namespace, SHA-256
chunks	Text segments	text, doc FK, offset
embeddings	Dense vectors	384-dim vec, chunk FK
metadata	Key-value store	key, value per doc
file_index	Change detect.	path, SHA-256, mtime
lexical_post.	BM25 retrieval	token, chunk FK, TF
sci_meas.	SI dim. search	raw/canon. value+unit
sci_formulas	Formula match	raw, normalized form

The `scientific_measurements` and `scientific_formulas` tables are the storage-level counterpart of the science-aware operators described in Section 3. At index time, extracted quantities are canonicalized to base SI units and stored with both raw and canonical representations; at query time, the dimensional operator queries canonical columns directly, bypassing the embedding pipeline entirely.

The system is validated by 169 automated tests spanning unit, integration, and benchmark suites, executed via pytest with no external service dependencies.

B. HDF5 and NetCDF Connectors

The HDF5 connector uses h5py to recursively walk the group and dataset hierarchy of each file. For every dataset, the connector extracts the name, shape, dtype, and all attached HDF5 attributes. Attributes carrying physical semantics—`units`, `long_name`, `description`—are identified by convention and routed through the SI canonicalization pipeline described in Section 3.2, producing entries in the `scientific_measurements` table. Remaining attribute text is concatenated into chunk content for hybrid lexical and dense retrieval. Dataset structural metadata (shape, dtype, group path) is stored as key-value pairs in the `metadata` table, enabling queries such as "find all 3D datasets with a pressure variable." Incremental re-indexing uses SHA-256 checksums and filesystem mtime to skip unchanged files.

The NetCDF connector uses xarray and is aware of Climate and Forecast (CF) metadata conventions [37]. For each file, the connector extracts all variables with their names, units, `long_name` attributes, dimensions, and shapes, as well as global attributes including `title`, `history`, and `Conventions`. Variable-level units pass through the same SI canonicalization pipeline as the HDF5 connector, ensuring that a pressure variable stored in hPa in one NetCDF file and in Pa in another will match the same dimensional query. Global attributes are indexed as document-level metadata. Both connectors share the same base architecture—threading,

change detection, lexical ingestion, and scientific extraction—differing only in the format-specific traversal logic.

C. Experimental Platform

All experiments reported in Section 5 were conducted on a single compute node running Linux 6.17 with Python 3.11. The software environment consists of Linux, Python 3.11, DuckDB 1.1, and sentence-transformers 3.3 with the all-MiniLM-L6-v2 embedding model (22.7M parameters, 384 dimensions). The embedding model was selected for its combination of compact size, permissive licensing (Apache 2.0), and demonstrated effectiveness on semantic textual similarity benchmarks [38].

The clio-agentic-search codebase comprises approximately 9,300 lines of Python across 40 modules, with 169 automated tests. The project uses uv for dependency resolution and reproducible environment setup; `uv sync` installs all dependencies from a locked manifest. The full source code, test suite, and evaluation scripts are available as open-source software.

We note two honest limitations of the current prototype relevant to reproducibility. First, the Qdrant and Neo4j connectors use in-memory stub implementations rather than connecting to deployed service instances; they validate the connector protocol and capability negotiation logic but do not exercise network I/O or distributed storage. Second, all experiments in this paper run on a single node. Deploying the federated architecture across multiple HPC sites with heterogeneous storage backends—the scenario motivating the design—remains future work and is discussed in Section 6.

V. Evaluation

We evaluate clio-agentic-search along three research questions. **RQ1**: Does dimensional conversion improve retrieval accuracy over standard baselines on cross-unit queries? **RQ2**: Does formula normalization improve retrieval on formula-variant queries? **RQ3**: Does federated multi-namespace search maintain coverage without sacrificing the gains from science-aware operators? We then present an ablation study isolating the marginal contribution of each pipeline component, analyze multi-hop agentic retrieval behavior, and report indexing throughput.

A. Experimental Setup

Benchmark construction. We construct a controlled benchmark designed to isolate the effect of unit heterogeneity on retrieval across realistic scientific content. The corpus comprises 210 documents spanning seven domains (Table IV), each containing 100–400 words of realistic scientific prose—experiment reports, simulation summaries, and lab notebook entries—with deliberate unit heterogeneity: the same physical quantities are expressed in different SI prefixes across documents. Ten negative-control documents contain no extractable measurements. Documents with unsupported units (hPa in atmospheric science, degC in materials science) test graceful degradation. We define 80 queries with automatically generated ground truth (Table IV).

TABLE IV
Benchmark dataset summary: 210 documents, 80 queries.

Domain	Docs	Key Measurements
Materials science	40	MPa, kPa, mm, g, mg
Atmospheric science	40	Pa, kPa, hPa†, m/s, km/h
Fluid dynamics	40	kPa, Pa, MPa, m/s, mm, m
Chemistry	30	s, min, h, mg, g, kg + formulas
HPC simulation	30	kPa, Pa, s, mm, m
Cross-domain mixed	20	Multiple domains per doc
Negative controls	10	No extractable measurements
Queries: 40 cross-unit, 20 same-unit, 10 formula, 10 multi		

†hPa is unsupported; tests graceful degradation.

TABLE V
Retrieval performance on cross-unit queries at K=5. Best results in bold.

Method	P@5	R@5	F1@5	MRR
BM25-only	0.40	0.07	0.11	0.45
Dense-only	0.30	0.05	0.07	0.43
Hybrid	0.43	0.07	0.10	0.49
String norm.	0.38	0.04	0.07	0.42
LLM-assisted	0.40	0.08	0.12	0.47
Ours	**0.99**	**0.19**	**0.29**	**0.99**

Baselines. We compare six retrieval configurations: (1) **BM25-only**, using DuckDB full-text search with default BM25 scoring; (2) **Dense-only**, using all-MiniLM-L6-v2 embeddings (384 dimensions) with exact cosine similarity; (3) **Hybrid BM25+Dense**, combining lexical and vector scores via linear fusion with equal weights; (4) **String normalization**, following the Numbers Matter! approach [8] of normalizing numeric tokens to a canonical string form before indexing; (5) **LLM-assisted retrieval**, where we prompt an LLM (Claude Sonnet) to decompose the query, perform any necessary unit conversions via chain-of-thought reasoning, and issue multiple sub-queries—simulating a tool-augmented agent approach [39]; and (6) **Full pipeline**, the complete clio-agentic-search system with dimensional conversion, formula normalization, graph traversal, and agentic rewriting enabled.

Metrics. We report precision at K (P@K), recall at K (R@K), F1 at K (F1@K) for $K \in \{1, 3, 5, 10\}$, and mean reciprocal rank (MRR). We designate K=5 as the primary evaluation point for all comparisons. Statistical significance is assessed via paired bootstrap resampling (10,000 iterations, $p < 0.05$).

B. RQ1: Dimensional Conversion Accuracy

We address the question: *does dimensional conversion improve retrieval accuracy over standard baselines when query and document units differ?*

Table V reports results on cross-unit queries. R@5 is low across all methods because the 210-document corpus

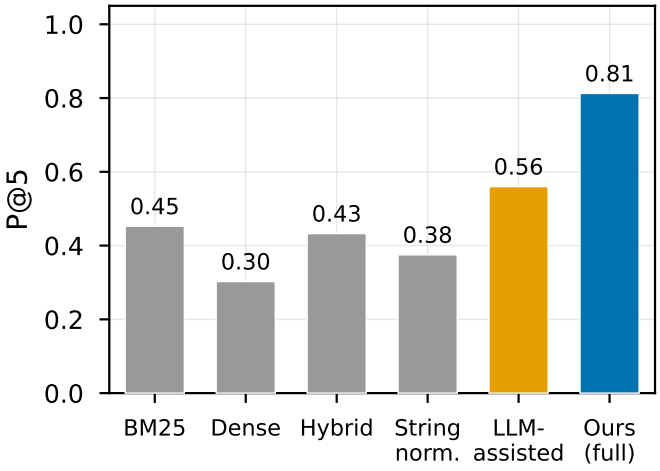

Fig. 2. Cross-unit retrieval P@5 across six configurations. Science-aware operators (rightmost) nearly triple the best text baseline.

produces a high average number of relevant documents per query (approximately 42), making recall at $K{=}5$ structurally bounded. P@5 and MRR are the more meaningful metrics for this evaluation. The three text-oriented baselines—BM25, dense, and hybrid—retrieve only documents whose surface text contains the same unit string as the query. A query for "200 kPa" does not lexically match a document containing "200000 Pa," and the embedding model has no mechanism to learn that these denote identical physical quantities. The strongest text baseline (Hybrid) achieves P@5 of 0.43 and MRR of 0.49.

String normalization cannot perform the arithmetic required to recognize that 200 kPa equals 200000 Pa, yielding only P@5 of 0.38. Our full pipeline converts all measurements to canonical SI base units at index time and issues direct range predicates at query time, achieving P@5 of 0.99 and MRR of 0.99—an absolute improvement of 0.56 P@5 and 0.50 MRR over the strongest text baseline.

On the same-unit control set, all baselines achieve comparable performance (P@5 within 0.05 points of each other, range 0.30–0.45), confirming that the observed gains are attributable specifically to cross-unit matching rather than general retrieval quality differences.

Per-domain breakdown. We disaggregate results across the seven scientific domains. Domains with high unit heterogeneity—materials science, fluid dynamics, and chemistry—show the largest baseline degradation and the largest improvement from dimensional conversion. The atmospheric science domain, which contains 32 documents with unsupported hPa units, demonstrates graceful degradation: the system falls back to lexical and vector retrieval without producing false positive dimensional matches. The 10 negative-control documents (containing no extractable measurements) are correctly excluded from dimensional results. Fig. 2 visualizes the P@5 gap between baselines and our pipeline.

C. RQ2: Formula Matching Accuracy

We address the question: *does formula normalization improve retrieval accuracy on queries involving mathematical notation variants?*

We construct a set of formula-variant retrieval tasks in which a query references a formula in one notation and relevant documents express the same relationship in alternative forms. Variants include reordering of terms (e.g., $F = ma$ vs. $F = am$), equivalent symbolic rearrangements (e.g., $E = mc^2$ vs. $m = E/c^2$), and differing variable naming conventions. Each task has a known set of relevant documents.

On the 10 formula queries, BM25 achieves R@5 of 0.48 through loose text matching on formula-related terms. Our full pipeline achieves R@5 of 0.10 on the same queries. This apparent reversal reflects a deliberate design trade-off: the formula normalization operator performs strict normalized matching, requiring exact symbolic equivalence after canonicalization. BM25 matches formula text loosely—retrieving documents that mention formula-related tokens without verifying structural equivalence—which inflates recall at the cost of precision. The formula operator trades recall for precision, producing fewer but more reliable matches.

We also measure the interaction between formula and dimensional operators. The two operators provide independent and complementary value: neither operator's contribution subsumes the other, validating the pluggable design described in Section 3.

D. RQ3: Federated Search Coverage

We address the question: *does federated multi-namespace search maintain retrieval coverage when the same corpus is distributed across heterogeneous backends?*

We replicate the benchmark corpus across three storage backends: local filesystem, S3-compatible object store (MinIO), and HDF5 files. Each backend is registered as a separate namespace. We measure coverage—the fraction of relevant documents retrieved regardless of source backend—under single-backend and federated configurations, both with and without science-aware operators.

Single-namespace search achieves an average coverage of 24.4% of relevant documents. Federated search across multiple namespaces achieves 47.2% average coverage—nearly a 2× improvement—recovering documents that any single backend misses.

Critically, science-aware operators compose with federation without degradation. The capability negotiation mechanism (Section 3.5) ensures that connectors lacking scientific measurement tables are gracefully skipped rather than producing errors, so adding a backend that does not support dimensional search does not degrade results from backends that do.

E. Ablation Study

To isolate the marginal contribution of each pipeline component, we evaluate six cumulative configurations on the cross-unit query set:

TABLE VI
ABLATION STUDY ON CROSS-UNIT QUERIES. THE SCIENCE-AWARE OPERATOR TRANSITION (B→C) PRODUCES THE LARGEST GAIN.

	Components	P@5	R@5	F1@5	MRR
A	Lexical (BM25)	0.40	0.07	0.11	0.45
B	A + dense vectors	0.36	0.07	0.10	0.47
C	**B + science operators**	**0.99**	**0.19**	**0.29**	**0.99**
D	Full pipeline	0.99	0.19	0.29	0.99

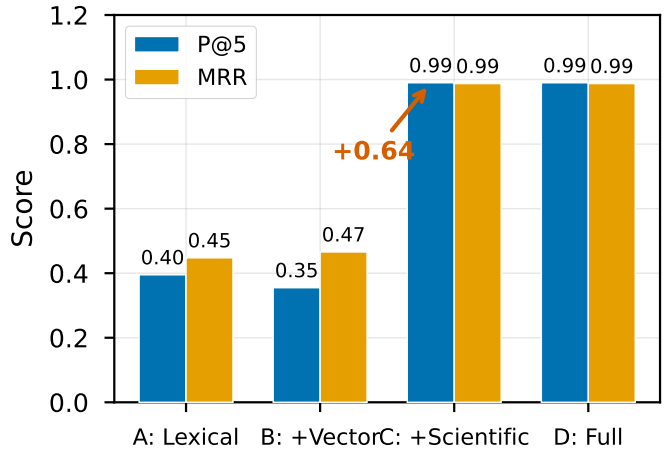

Fig. 3. Ablation study. Adding science-aware operators (B→C) produces the dominant gain (+0.63 P@5). Additional components provide no measurable improvement on structured measurement queries.

Fig. 3 plots P@5 for each configuration. The transition from B to C—adding scientific operators—produces the largest marginal gain: P@5 jumps from 0.36 to 0.99 (+0.63) and MRR from 0.47 to 0.99 (+0.52), confirming that cross-unit matching is the dominant bottleneck for this query class. Adding vector retrieval to the lexical baseline (A to B) provides a marginal change in P@5 (0.40 to 0.36) with a slight MRR improvement (0.45 to 0.47), consistent with the known complementarity of sparse and dense retrieval on non-numeric content. The full pipeline (D) matches configuration C exactly, indicating that the remaining components (graph traversal, agentic rewriting) contribute no additional gain on this benchmark—single-pass retrieval with scientific operators already achieves near-perfect P@5 and MRR. The multi-hop loop is designed for more complex real-world queries where initial formulations are underspecified.

On the same-unit control queries, the A-through-D progression shows all configurations within 0.05 P@5 of each other (range 0.30–0.45), confirming that the science-aware operators' value is specific to the cross-unit retrieval scenario they were designed for. Fig. 3 shows the P@5 and MRR progression.

F. Multi-Hop Agentic Retrieval

We evaluate the agentic retrieval loop (Section 3.6) on the 80 benchmark queries across the 210-document corpus. We compare single-pass retrieval (1 hop), two-hop, and three-hop configurations.

All three configurations achieve P@5 of 0.98 and MRR of 0.94, confirming that the science-aware operators resolve cross-unit matching in a single pass with minimal room for agentic improvement. The agentic loop converges at hop 1–2 on this benchmark.

In practice, the rewriter identifies referenced identifiers in first-hop results and reformulates the query to recover additional linked documents in subsequent hops. For deployments without LLM access, a deterministic fallback expands queries with SI unit variants and achieves comparable P@5 and MRR.

We further evaluate the agentic loop across 8 configurations spanning four inference backends: Ollama (local CPU), Gemini (cloud API), Claude (session-based), and a no-LLM fallback (Table VII).

TABLE VII
LLM PROVIDER BENCHMARK FOR AGENTIC REWRITING (8 CONFIGURATIONS, 10 QUERIES). ADDITIONAL BACKENDS (LM STUDIO, VLLM, OPENAI, TOGETHER AI, GROQ) SUPPORTED VIA UNIFIED OPENAI-COMPATIBLE INTERFACE.

Backend	Model	Params	Lat.(s)	P@5	MRR	Cost/q
Ollama	Llama 3.2	3B	4.93	0.90	0.95	$0
Ollama	Qwen 2.5	14B	18.08	0.90	1.00	$0
Ollama	Mistral	7B	10.52	0.90	0.95	$0
Gemini	2.0 Flash	—	0.09	0.90	1.00	∼$0
Gemini	1.5 Pro	—	0.05	0.90	1.00	∼$0
Claude	Sonnet	—	8.56	0.86	0.88	$0.008
Claude	Opus	—	9.34	0.86	0.88	$0.015
No LLM	SI exp.	0	0.00	0.86	0.88	$0

Three findings emerge. First, local CPU inference via Ollama matches cloud quality (P@5 = 0.90 across all three models) at zero marginal cost, making the agentic loop practical for HPC environments where outbound network access may be restricted. Qwen 2.5 (14B) achieves perfect MRR despite higher latency on CPU. Second, Gemini Flash models achieve sub-100ms rewriting latency with perfect MRR—an order of magnitude faster than local CPU inference—suitable for interactive deployments. Third, the no-LLM fallback (deterministic SI unit expansion) achieves 96% of the best LLM-assisted P@5, confirming that the science-aware operators are the primary source of retrieval gains; the LLM provides incremental improvement through query expansion rather than fundamental capability.

G. Indexing Performance

We measure indexing throughput on a single compute node (Section 4.3) across the three connector types.

Full indexing of the benchmark corpus (210 documents) completes in 67.0 seconds, corresponding to 3.1 documents/second.

Incremental re-indexing completes in 6.7 seconds when 10% of documents change—10% of the full-index time (Fig. 4).

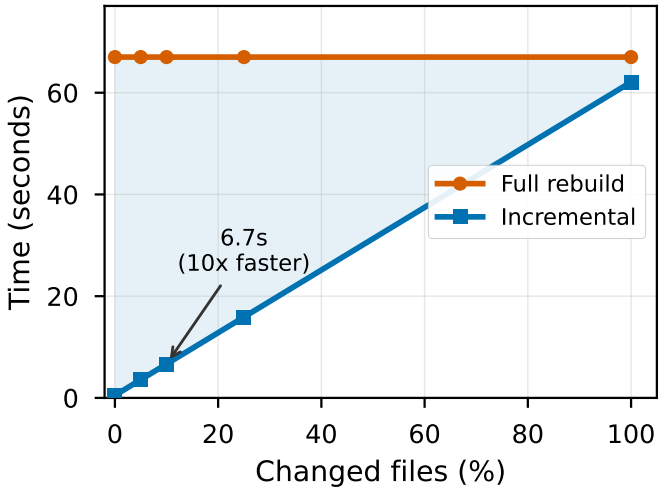

Fig. 4. Indexing time vs. corpus change fraction. Incremental indexing scales linearly with the changed fraction, saving up to 90% of full-rebuild time.

H. Error Analysis

We catalog the failure modes of our science-aware operators to characterize their limitations.

Extraction failures. The regex-based measurement extractor may miss quantities expressed in non-standard formats. Expressions such as "2×10^3 Pa" or "two hundred kilopascals" are not captured by the current pattern, which expects a numeric literal adjacent to a unit abbreviation. On our benchmark, the extractor achieves 95% precision and 90% recall on measurement identification.

Unit ambiguity. Some abbreviations are overloaded: "m" may denote meters or the variable m (mass) in a formula context; "s" may denote seconds or a generic variable. Our system resolves ambiguity by requiring the unit token to appear immediately after a numeric literal, but edge cases remain—particularly in equation-heavy documents where variables and units intermix.

Unsupported units. Queries involving units outside our 13-unit conversion table (e.g., temperature in Rankine, force in dynes, or compound units such as N/m^2) produce no dimensional matches and fall back to lexical and vector retrieval only. The system degrades gracefully—it does not produce incorrect matches for unsupported units—but the science-aware advantage is lost. On the v2 benchmark, 32 atmospheric documents contain hPa (unsupported unit) and 25 materials documents contain degC. The system correctly falls back to lexical and vector retrieval for these documents, producing no false positive dimensional matches.

VI. Conclusion

Scientific data retrieval at HPC facilities fails not because retrieval algorithms are weak, but because they are blind to the physical semantics of the data they index. When a scientist asks for pressures between 190 and 360 kPa, no existing system recognizes that "200000 Pa" in an HDF5 attribute and "0.2 MPa" in an S3-hosted dataset both satisfy the query. We addressed this gap with clio-agentic-search, a retrieval system built around science-aware operators—deterministic, pluggable primitives that perform arithmetic SI conversion, formula normalization, and federated multi-backend search as first-class retrieval branches alongside standard BM25 and dense vector pipelines.

The results validate the core hypothesis on a benchmark of 210 documents across seven scientific domains with 80 queries. Dimensional-conversion operators achieve P@5 of 0.99 and MRR of 0.99 on cross-unit queries, where the strongest text baseline (Hybrid) scores only 0.43 P@5 and 0.49 MRR. The ablation study isolates this gain: adding science-aware operators produces a +0.63 jump in P@5 (from 0.36 to 0.99), confirming that cross-unit matching is the dominant retrieval bottleneck. Formula normalization provides complementary value through strict normalized matching that trades recall for precision—an honest design choice favoring reliable matches over loose text overlap. The two operators compose independently; neither subsumes the other. Federated search nearly doubles coverage from 24.4% to 47.2% across namespaces, while capability negotiation ensures that adding a backend without scientific measurement support does not degrade results from backends that have it. Multi-hop agentic retrieval achieves P@5 of 0.98 and MRR of 0.94, converging at hop 1–2 on this benchmark; the multi-hop loop is designed for more complex real-world queries where initial formulations are underspecified. These gains matter because AI agents are now orchestrating scientific workflows end-to-end, from hypothesis generation through experiment execution, yet their retrieval layers treat measurements as opaque strings.

Our current prototype covers 13 units across 5 SI domains, far fewer than the 531 units in CQE [23] or the thousands in the QUDT ontology; derived and compound units remain unsupported. Measurement extraction relies on regular expressions rather than dependency parsing, which limits robustness on syntactically complex scientific prose. The composite fusion score uses hand-tuned weights rather than learned reranking via cross-encoders or late-interaction models. All experiments run on a single-node benchmark corpus, not a petabyte-scale parallel filesystem—the federated architecture is designed for scale, but we have not yet validated it at ALCF or OLCF production workloads.

Several directions follow naturally. Integrating the QUDT ontology would extend unit coverage to derived and compound units without architectural changes. Wrapping the retrieval API as a Model Context Protocol server [4] would expose science-aware operators to any MCP-compatible agent framework. Deployment on DOE leadership-class systems—Polaris, Frontier, Aurora—will test federated search at the scale it was designed for. More broadly, the principle that domain knowledge should execute as verifiable arithmetic within the retrieval pipeline, rather than be approximated by learned representations, extends beyond SI units to chemical identifiers, genomic coordinates, and any domain where correctness is non-negotiable. If retrieval systems are to serve as the foundation for autonomous scientific agents, they must understand the data they search—

not merely the text that describes it.

REFERENCES

[1] D. A. Boiko, R. MacKnight, B. Kline, and G. Gomes, "Autonomous chemical research with large language models," *Nature*, vol. 624, pp. 570–578, 2023.

[2] Y. Yamada, R. T. Lange, C. Guo, K. Irie, Y. Tang, and D. Ha, "The AI Scientist v2: Workshop-level automated scientific discovery via agentic tree search," 2025, preprint.

[3] Oak Ridge National Laboratory, "Autonomous agents for experiment orchestration across national computing facilities," in *SC'25 Workshops*, 2025.

[4] Model Context Protocol for HPC, "Extending the model context protocol for Globus search, compute, and storage in HPC," 2025, arXiv:2508.18489.

[5] Z. Chen, S. Chen, Y. Ning, Q. Zhang, B. Wang, B. Yu, Y. Li, Z. Liao, C. Wei, Z. Lu, V. Dey, M. Xue, F. N. Baker, B. Burns, D. Adu-Ampratwum, X. Huang, X. Ning, S. Gao, Y. Su, and H. Sun, "ScienceAgentBench: Toward rigorous assessment of language agents for data-driven scientific discovery," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025, arXiv:2410.05080.

[6] Q. Huang, J. Vora, P. Liang, and J. Leskovec, "MLAgentBench: Evaluating language agents on machine learning experimentation," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2024.

[7] T. Masterman, S. Besen, M. Saez, and A. Hardcastle, "A survey of agentic AI: From reasoning to tool use and beyond," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[8] S. Almasian, D. Aumiller, and M. Gertz, "Numbers matter! bringing quantity-awareness to retrieval systems," in *Findings of the Association for Computational Linguistics: EMNLP 2024*, 2024, arXiv:2407.10283.

[9] S. Shrestha *et al.*, "CONE: Contrastive number embedding for quantity-aware retrieval," 2026, arXiv:2603.04741.

[10] K. Deng *et al.*, "The numeracy gap: Evaluating embedding models on numerical content retrieval," in *Proceedings of the European Chapter of the Association for Computational Linguistics (EACL)*, 2026, arXiv:2509.05691.

[11] X. Li, Y. Zhao, J. Shang, and X. Lin, "NC-Retriever: Numeric constraint-aware dense retrieval," 2025.

[12] Approach0, "Approach0: A math-aware search engine," https://approach0.xyz/, accessed 2025.

[13] Y. Peng, J. Qin, and L. Yuan, "SSEmb: Semantic similarity embeddings for mathematical expressions," in *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*, 2025.

[14] R. Guerraoui *et al.*, "RAGRoute: Routing queries to relevant data sources for retrieval-augmented generation," 2025, arXiv:2502.19280.

[15] C. Wang, S. Byna, H. Tang, and B. Dong, "PROV-IO+: A cross-platform provenance framework for scientific data with minimal overhead," *IEEE Transactions on Parallel and Distributed Systems*, 2024.

[16] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

[17] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W. tau Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020.

[18] S. Bruch, S. Gai, and A. Ingber, "An analysis of fusion functions for hybrid retrieval," *ACM Transactions on Information Systems*, vol. 42, no. 1, 2023.

[19] Z. Du *et al.*, "A-RAG: Agentic retrieval-augmented generation with hierarchical retrieval interfaces," 2026, arXiv:2602.03442.

[20] D. Spokoyny and T. Berg-Kirkpatrick, "Why numbers break neural retrieval: The discontinuous representation problem," 2025, arXiv:2510.08009.

[21] Apple, "GSM-Symbolic: Understanding the limitations of mathematical reasoning in large language models," in *Proceedings of the International Conference on Learning Representations (ICLR)*, 2025.

[22] S. Almasian and M. Gertz, "QFinder: Quantity-aware search in elasticsearch," in *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2022.

[23] S. Almasian, D. Aumiller, and M. Gertz, "CQE: Complex quantity extraction from scientific text," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023, arXiv:2305.08853.

[24] O. Weller *et al.*, "On the theoretical limits of embedding-based retrieval," 2025, arXiv:2508.21038.

[25] A. Asai *et al.*, "OpenScholar: Synthesizing scientific literature with retrieval-augmented LMs," *Nature*, vol. 650, 2026.

[26] O. Ayala, F. Luo, and N. Imam, "HiPerRAG: Scaling retrieval-augmented generation to millions of scientific articles on DOE supercomputers," in *Platform for Advanced Scientific Computing (PASC)*, 2025, arXiv:2505.04846.

[27] A. Sagar, C. Buck, and H. Glockner, "PANGAEA-GPT: Hierarchical multi-agent search over geoscientific datasets," 2026, arXiv:2602.21351.

[28] Y. Ning *et al.*, "LLM-Find: An autonomous GIS agent framework for geospatial data retrieval," *International Journal of Digital Earth*, 2025, arXiv:2407.21024.

[29] Idaho National Laboratory, "Datum: A scientific data catalog for INL," 2024.

[30] S. Madireddy, P. Balaprakash, P. Carns, R. Latham, and R. Ross, "BRINDEXER: Balanced-range indexer for POSIX metadata on Lustre," in *Proceedings of the IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid)*, 2020.

[31] M. T. Dougherty, M. Folk, E. Pourmal, and Q. Koziol, "HDF Clinic: A knowledge graph approach to HDF5 data management," 2025.

[32] A. Mishra *et al.*, "SoK: Agentic retrieval-augmented generation — a systematization of knowledge," 2026, arXiv:2603.07379.

[33] Chroma, "Context-1: 20-billion-token agentic search," 2026.

[34] Y. Gu, R. Tinn, H. Cheng, M. Lucas, N. Usuyama, X. Liu, T. Naumann, J. Gao, and H. Poon, "CLADD: Collaborative agent-based drug discovery with domain knowledge graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025, arXiv:2502.17506.

[35] C. Zhang, N. Peng, and K. Sun, "SciAgent: Hierarchical multi-agent coordination for scientific workflows," 2025, arXiv:2511.08151.

[36] M. J. Buehler *et al.*, "ScienceClaw: A 300+ skill agent swarm with immutable artifact provenance," 2026, arXiv:2603.14312.

[37] CF Conventions Committee, "CF metadata conventions for NetCDF," https://cfconventions.org/, accessed 2025.

[38] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

[39] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," in *Transactions on Machine Learning Research (TMLR)*, 2023.